

「GPU 공동 사용 플랫폼」HAMi 기술검증 결과보고서

컴퓨터공학전공 2020112028 정상원

1. 검증 개요

1.1 배경

「구현 방식 검토 결과보고서」는 네 가지 구현 방식을 비교한 끝에 가상 인스턴스 임대 방식을 택했다. 작업 스케줄링 방식이 다룰 수 있는 최소 단위는 GPU 한 장이고, 기증받은 장비가 열 몇 장인 환경에서 한 강의실의 학생 마흔 명에게 그래픽카드를 한 장씩 배정하는 일은 시간을 더 들이거나 설정을 바꾼다고 해결되지 않으므로, 그래픽카드를 분할해 배정하는 방식만이 이 규모를 감당한다. 따라서 선택한 구현 방식은 한 가지 기술적 전제에 의존하며, 그 전제가 성립하지 않으면 앞선 보고서의 결론도 성립하지 않는다.

전제를 뒷받침할 기술로 지목한 HAMi-core 는 CUDA 런타임과 CUDA 드라이버 사이에서 호출을 가로채는 사용자 공간 라이브러리다. 커널 수정도 드라이버 패치도 필요 없다는 점에서 우리 여건에 맞지만, CNCF 인큐베이팅 단계에 들어간 지 얼마 되지 않았고 통상 Kubernetes 장치 플러그인으로 배포되며 우리가 쓰려는 단독 Docker 구성은 공식 문서의 예제가 한 줄뿐이다. 실제 하드웨어에서 안정적으로 동작하는지를 확인한 사례는 찾지 못했다.

1.2 검증 대상 가정

본 검증은 다음 한 문장이 성립하는지를 확인하고자 한다.

물리 GPU 한 장을 여러 개의 격리된 뭉치로 나누어 각 뭉치에 VRAM 상한을 강제할 수 있고, 그 안에서 실제 딥러닝 작업이 정확하게 실행되며, 한 사용자가 다른 사용자의 작업을 망가뜨릴 수 없다.

이 문장의 네 요소인 분할, 강제, 정확성, 격리가 각각 검증 항목이 된다. 어느 하나가 성립하지 않으면 플랫폼의 설계를 다시 잡아야 하므로, 검증 과정에는 미리 정한 중단 조건을 두어 그런 결과가 나오면 이후 단계를 진행하지 않고 보고하도록 했다.

1.3 검증 범위와 한계

검증 대상은 자원 분할 계층에 한정한다. REST API 와 웹 인터페이스, 인증, 스케줄링 정책, 크레딧 회계, 저장소 동기화, 접속 라우팅은 모두 제외했다. 성능 예측 역시 별개의 연구 주제이므로 다루지 않았고, HAMi 를 Kubernetes 없이 단독으로 쓰는 구성을 전제했으므로 Kubernetes 관련 사항도 포함하지 않았다.

검증에 사용한 장비가 GPU 한 장짜리 단일 서버라는 점에서 오는 한계가 두 가지 있다. 하나는 여러 장의 그래픽카드에 걸친 작업으로, 컨테이너 하나가 서로 다른 물리 장치에 각각 뭉치를 갖는 경우와 두 장 이상에 걸친 분산 학습의 정확성은 이 장비에서 답할 수 없다. 다른

하나는 구형 그래픽카드로, Pascal 이전 세대에서의 동작은 해당 장비가 없어 확인하지 못했다. 두 항목을 어떻게 처리했는지는 4.6 에서 다룬다.

2. 검증 방법

2.1 검증 환경

구분	내용
GPU	NVIDIA GeForce RTX 4080 SUPER, 16 376 MiB, compute capability 8.9, 디스플레이 미 연결
드라이버	580.178.04 (CUDA 13.0)
CPU / RAM	Intel i7-9700KF 8 코어, 31 GiB
OS / 커널	Ubuntu 22.04.5 LTS, 5.15.0-190-generic
컨테이너	Docker 29.1.3, NVIDIA Container Toolkit 1.20.0, cgroup v2
HAMi-core	커밋 f01e9f23, 2026-08-31, 상위 저장소의 빌드 절차 그대로
작업 이미지	pytorch/pytorch:2.13.0-cuda13.0-cudnn9-runtime 기반, PyTorch 2.13.0+cu130

HAMi-core 에는 릴리스 태그가 없어 특정 커밋을 고정해 사용했으며, 제어에 쓰는 환경 변수는 공개 문서를 따르지 않고 소스에서 직접 추출해 정리했다(별첨 1).

2.2 측정 방법

측정의 기준선은 호스트 쪽 NVML 이다. HAMi 는 NVML 질의도 가로채 컨테이너 안에서는 가상화된 값을 돌려주므로, 컨테이너 내부에서 읽은 수치는 어떤 경우에도 근거로 삼지 않았다. 호스트에서 100 밀리초 간격으로 표본을 수집하고 프로세스별 사용량을 cgroup 경로를 통해 컨테이너에 대응시켰다.

메모리를 말할 때는 네 가지 값을 구분한다. PyTorch 가 살아 있는 텐서에 실제로 쓴 양, 캐싱 할당자가 드라이버로부터 확보해 쥐고 있는 양, 드라이버가 해당 프로세스 몫으로 계산하는 양, 그리고 그래픽카드 전체에서 쓰이는 양이 각각 다르며, 세 번째와 두 번째의 차이가 CUDA 컨텍스트와 라이브러리 작업 공간의 고정 비용에 해당한다. 본문에서 별도 표시 없이 쓰는 수치는 세 번째 값이다.

정확성은 학습이 끝난 시점의 모델 파라미터 전체에 대한 SHA-256 해시로 판정했다. 손실 값이나 정확도가 비슷한 수준인지를 보는 대신 비트 단위로 일치하는지를 확인한 것이며, 이를 위해 난수 시드와 결정론적 알고리즘, 워커 시드까지 고정한 재현 환경을 각 작업에 적용했다. Stable Diffusion 은 생성된 이미지 픽셀의 해시로, GPT-2 는 평가 혼란도로 대신했다. 측정 도구 자체도 쓰기 전에 검증하여, 정확히 2 048 MiB 를 할당해 유지하는 프로브를 실행해 표본기가 2 048.0 MiB 의 차이를 보고하는지 확인한 뒤에야 이후 측정에 사용했다.

2.3 차수별 구성

검증은 세 차례에 나누어 수행했고, 각 차수는 앞 차수의 결과를 전제로 한다. 1차에서는 기본 동작을 다루어, 라이브러리를 빌드해 상한이 실제로 강제되는지, 상한 아래에서 실제 작업이 정확하게 실행되는지, 두 사용자가 서로의 작업에 영향을 줄 수 있는지, 실패했을 때 어떤 형태로 나타나는지를 확인하고, 마지막에 배정과 실행과 회수를 담당하는 실행기를 구성해 세 시간 동안 연속 운전했다. 2차에서는 규모를 다루었는데, 1차의 동시 실행 검증이 모두 두 개짜리 조합이었던 반면 플랫폼이 전제하는 것은 그래픽카드 한 장에 여덟 명이므로, 사용자 수를 늘려 가며 성능이 어떻게 나뉘는지, 여덟 명이 동시에 시작해도 되는지, 그래픽카드 한 장에 몇 개의 몫을 실제로 담을 수 있는지, 권한 없는 사용자가 무엇을 할 수 있는지를 차례로 확인했다. 3차에서는 남은 항목을 다루어, 여러 장에 걸친 작업에서 통신 라이브러리가 상한을 벗어나지 않는지, 세션을 12 시간 유지해도 문제가 없는지, 사용자가 직접 CUDA 확장을 컴파일할 수 있는지를 확인했다.

3. 검증 결과

세 차례의 검증에서 얻은 결과를 차수가 아니라 주제별로 나누어 정리한다. 각 수치가 어느 차수에서 나왔고 어떤 조건에서 측정되었는지는 별첨 2부터 별첨 4까지에 실었다.

3.1 메모리 격리의 실효성

상한은 실제로 강제된다. 설정한 값에 대해 컨테이너 안에서 실제로 쓸 수 있는 양은 언제나 256 MiB 적었고, 그래픽카드에서 실제로 소모되는 양은 언제나 32 MiB 많았다. 두 차이는 2 GiB 부터 8 GiB 까지 어느 설정에서도 같은 크기로 나타났으므로 비율이 아닌 고정 비용이며, 따라서 배정 계산에 그대로 반영할 수 있다. 사용 가능량이 줄어드는 것은 HAMi가 컨텍스트 몫으로 예산에서 250 MiB를 미리 제외하기 때문이고, 물리 소모량이 늘어나는 것은 컨텍스트가 실제로는 290 MiB를 사용하는데 계산에는 250만 반영되기 때문이다.

상한 아래에서 실제 작업은 정확하게 실행된다. CIFAR-10 위의 ResNet-18 부터 Imagenette 위의 ResNet-50, SST-2 위의 BERT, WikiText-2 위의 GPT-2, 그리고 Stable Diffusion 추론까지 다섯 종을 분할된 몫 안에서 돌렸을 때 스무 번의 실행이 모두 분할하지 않은 기준값과 비트 단위로 일치했다. 가로채기 계층을 적용할 때 발생하는 비용은 처리량 기준 1% 미만이고, 상한을 걸었다고 해서 추가로 발생하는 비용은 측정되지 않았다.

상한을 넘으면 파이썬에서 잡을 수 있는 예외가 발생하며 프로세스는 종료되지 않는다. 예외를 잡은 뒤 계산을 이어 가고 메모리를 반환한 다음 다시 할당하는 일까지 정상적으로 이루어졌고, 30 초 동안 459 회, 75 초 동안 1096 회를 연달아 넘겨 봐도 드라이버 수준 오류는 한 건도 나오지 않았다. 검증 기간 전체에 걸쳐 Xid 는 한 건도 기록되지 않았다.

상한은 프로세스가 아니라 몫에 걸린다. 컨테이너 하나 안에서 자식 프로세스 셋을 띄워 각각 1536 MiB 씩 요구하게 하면 셋째가 거절되므로 포크로 우회할 수 없고, 여러 장에 걸친 작업에서 쓰이는 NCCL 의 통신 버퍼도 마찬가지로 예산 안에서 계산되어 4 GiB 몫을 3.5 GiB 까지 채운 뒤 초기화하면 정확히 4 GiB 선에서 거절된다. 같은 조건에서 라이브러리를 붙이지 않은 대조군은 5 GiB 까지 제한 없이 자랐다. 사용자가 직접 작성한 CUDA 확장이 드라이버 함수를 직접 호출해 메모리를 요구하는 경우에도 상한이 지켜졌으며, cudart 를 정적으로 링크한 독립 실행 파일 역시 드라이버 라이브러리만은 동적으로 호출하기 때문에 가로채기를 우회하지 못했다.

3.2 연산 배분 제한의 특성

CUDA_DEVICE_SM_LIMIT 은 이름과 달리 연산 능력의 지분을 뜻하지 않는다. 큰 행렬곱을 연속으로 실행하는 작업에 50 %를 걸어도 그래픽카드 성능의 92 %가 나오고, 제한의 효과가 뚜렷해지는 것은 25 % 아래부터다. 반면 작은 커널을 많이 실행하고 동기화가 잦은 ResNet-18 학습은 50 %에서 54 %로 거의 비례해 줄어든다. 커널 실행 요청을 지연시켜 속도를 낮추는 방식이라 작업의 커널 구성에 따라 같은 숫자가 다른 결과를 낳는다.

제한이 걸린 작업은 그래픽카드가 비어 있어도 대기 상태를 유지한다. 제한 없는 작업과 30 %로 제한한 작업을 함께 실행하면 제한 없는 쪽이 먼저 끝난 뒤에도 제한이 걸린 쪽은 계속 대기하므로 둘을 합친 완료 시각이 같은 크기의 균등한 한 쌍보다 오히려 늦지만, 경합 상황에서 우선순위를 주는 수단으로는 정확히 작동해 소유 연구실의 작업을 손님의 작업보다 앞세우는 데 쓸 수 있다.

제한을 걸지 않으면 시간 분할은 사용자가 아니라 커널 실행 시간을 기준으로 나눈다. 커널 하나가 오래 도는 ResNet-50 을 ResNet-18 다섯 개와 함께 실행하면 ResNet-50 은 3.31 배 느려지는데 그치는 반면 ResNet-18 쪽은 5.9 배 가까이 느려지므로, 무거운 작업이 균등 배분보다 많은 연산을 배정받는 셈이다.

3.3 다중 사용자 동시 실행

사용자 수를 둘, 넷, 여섯, 여덟, 아홉으로 늘려 가며 측정한 결과 개별 작업의 지연은 사용자 수에 반비례해 늘어났고 그 오차는 4 % 안이었다. 그래픽카드 전체의 처리량은 오히려 사용자가 많을수록 조금씩 올라가 여덟 명에서 0.993 에 이르렀는데, 작업 하나가 남기는 빈 시간을 다른 작업이 메우기 때문이다. 가장 운이 나쁜 사용자와 가장 좋은 사용자의 차이도 5 %를 넘지 않았고 모든 작업이 비트 단위로 정확했다. 한계를 정하는 것은 시간 분할이 아니라 메모리여서, 열 번째 컨테이너는 그래픽카드가 물리적으로 가득 찬 상태에서 시작하다 깨끗한 메모리 부족으로 종료했고 이미 실행 중이던 아홉은 아무 영향도 받지 않았다.

동시 사용자 수	개별 지연 (평균)	이상값	합산 처리량	편차	정확성
2	2.00	2	0.971	1.002	전부 일치
4	3.90	4	0.978	1.016	전부 일치
6	5.78	6	0.985	1.028	전부 일치
8	7.67	8	0.993	1.040	전부 일치
9	8.62	9	0.991	1.049	전부 일치
10	아홉은 정상, 열째가 시작 시 실패	—	—	—	아홉 일치

그래픽카드 한 장에 담을 수 있는 뭉치의 수는 실측한 최소 크기에서 계산된다. 그래픽카드가 실제로 내줄 수 있는 양은 총량이 아니라 드라이버가 예약한 455 MiB 를 제외한 여유량 15 904 MiB 이며, 뭉치 하나가 차지하는 물리량은 설정값에 32 MiB 를 더한 값이므로 여기에 여유를 두어 64 를 더한 값으로 계산한다.

그래픽카드당 인원	설정값	내부 가용	수용 가능한 작업
8	1 920 MiB	1 664	ResNet-18 계열 실습
6	2 560	2 304	동상
4	3 904	3 648	BERT-base 미세 조정
3	5 184	4 928	Stable Diffusion 추론
2	7 872	7 616	ResNet-50, GPT-2
1	15 808	15 552	측정한 전부

여기서 2의 거듭제곱으로 등급을 잡으면 같은 그래픽카드에서 한 자리씩 줄어드는데, 4 GiB 등급으로는 세 명만 들어가지만 3 904 MiB로 잡으면 네 명이 들어가고 2 GiB 등급으로 일곱 명이던 것이 1 920 MiB에서 여덟 명이 된다. 각 등급은 해당 작업으로 실제로 가득 채워 실행해 확인했다.

동시 시작도 문제가 되지 않았다. 여덟 개를 같은 순간에 실행했을 때 초기화 실패는 없었고 전부 13초 안에 학습에 들어갔으며, 오히려 2초씩 간격을 두면 마지막 컨테이너가 학습을 시작하는 시점이 28초로 늦어졌다. 열여섯 개를 한꺼번에 제출했을 때도 실행기는 메모리가 허락하는 여덟 개만 동시에 돌렸고 배정 총량이 용량을 넘긴 표본은 한 건도 없었다.

이웃 작업이 자원을 과도하게 요구해도 격리는 유지된다. 4 GiB 뚫 안에서 메모리 부족이 날 때까지 할당을 반복하는 경우, 3초짜리 커널을 연속 실행하는 경우, 실행 중 강제 종료되는 경우, 라이브러리를 적용하지 않은 채 그래픽카드 전체를 점유하는 경우 네 가지를 차례로 붙여 실행했을 때, 대상 작업의 결과는 모두 비트 단위로 정확했다. 처리 속도는 낮아지지만 결과는 달라지지 않는다.

3.4 장시간 운영 안정성

실행기를 세 시간 동안 연속 운전하여 204건의 작업을 처리한 결과 전부 성공했고, 매 회차마다 용량이 정확히 원래 값으로 돌아왔으며, 세 시간 뒤 그래픽카드의 메모리 사용량은 시작 전과 같은 17 MiB였다. 남은 프로세스도, 새어나간 용량도 없었다.

세션을 장시간 유지하는 경우도 확인했다. 4 GiB 뚫을 12시간 동안 유지하면서 10분마다 한번씩 작업을 실행하는 컨테이너와 컨텍스트만 유지하는 컨테이너를 함께 두고, 여덟 개 시점에서 상한이 그대로인지와 새로 돌린 학습의 결과가 처음과 같은지를 확인한 결과다.

경과 시간	강제된 상한	학습 결과 해시	새 프로세스가 본 여유
0 시간	3 456 MiB	ce935317dcc2	3 469 MiB
2 시간	3 456	동일	3 469
6 시간	3 456	동일	3 469
10 시간	3 456	동일	3 469
11.8 시간	3 456	동일	3 469

열두 시간 동안 움직인 값이 없다. 상한도, 회계도, 결과도 그대로였고 컨테이너의 메모리 사용량은 첫 작업에서 한 번 올라간 뒤 끝까지 고정이었으며, Xid 는 0 이었고 두 컨테이너가 끝난 뒤 그래픽카드는 17 MiB 로 돌아왔다.

작업이 강제로 종료되는 경우에도 회수는 빠르다. 학습 중인 컨테이너를 강제 종료하면 0.22 초 만에 메모리가 반환되고 곧바로 다음 몫을 온전히 배정할 수 있다. 컨테이너를 만들고 없애는데 드는 시간은 회당 3.8 초 정도로, 대부분 컨테이너 시작과 CUDA 초기화가 차지한다.

3.5 사용자 작업 환경

권한 없는 사용자로도 통상적인 작업이 가능하다. 읽기 전용 이미지 위에서 uid 를 고정하고 모든 권한을 제거한 상태에서도 사용자 영역 패키지 설치, Jupyter Lab 과 TensorBoard 실행, 공유 모델 캐시 사용, 몫 한도까지의 GPU 메모리 할당이 모두 정상적으로 동작했다. 시스템 패키지 설치와 이미지 수정은 되지 않으므로, 필요한 패키지를 미리 담은 이미지와 사용자별 홈 볼륨을 제공하는 방식이 전제된다.

다만 홈 디렉터리를 어디에 두느냐가 결정적이다. Docker 가 기본으로 붙이는 임시 파일 시스템은 실행 권한이 없어, 여기에 홈을 두면 컴파일된 확장을 포함한 패키지가 설치되는 되지만 실행되지 않는다. Jupyter 는 pyzmq 에서, TensorBoard 는 grpc 에서 각각 세그먼트를 매핑하지 못했다는 알아보기 어려운 오류로 죽는다. 실행 가능한 볼륨을 홈으로 주면 두 가지 모두 정상 동작한다.

CUDA 확장을 직접 컴파일해야 하는 사용자를 위해서는 개발용 이미지가 필요하다. 컴파일러를 포함한 이미지 위에서 같은 무권한 조건을 적용했을 때 사용자가 직접 CUDA 커널을 컴파일해 22 초 만에 정상 실행했고, 그 확장이 드라이버 함수를 직접 불러 메모리를 요구해도 상한이 지켜졌으며, 우회 경로도 모두 그대로 막혀 있었다. 대신 용량이 늘어나, 실행용 이미지 6.3 GB 에 비해 개발용은 21.3 GB 다.

3.6 확인된 결함

세 차례의 검증에서 세 가지 결함을 찾았다. 첫째는 환경변수를 통한 상한 우회로, 라이브러리를 LD_PRELOAD 로만 주입하면 사용자가 그 변수를 지우고 프로그램을 실행하는 것으로 상한이 사라져 4 GiB 몫에서 6 GiB 를 할당하는 데 성공했으나, /etc/ld.so.preload 에 읽기 전용으로 넣으면 변수를 지워도 자기 것으로 복사한 로더를 써도 정적으로 링크한 바이너리를 써도 상한이 유지된다. 둘째는 비동기 할당자와의 충돌로, PyTorch 의 cudaMallocAsync 백엔드를 쓰면 HAMi 가 스트림 캡처 중 장치 번호를 잘못 계산해 CUDA 그래프 캡처가 실패하는데, 기본 할당자에서는 CUDA 그래프와 컴파일 최적화가 모두 정상 동작하고 상한도 지켜지므로 비동기 백엔드를 제공하지 않으면 된다. 셋째는 동시 시작시의 경합으로, 두 컨테이너가 같은 순간에 시작하고 그중 하나가 곧바로 큰 메모리를 요구하면 다른 하나가 이웃의 사용량을 자기 것으로 잘못 읽어 자기 컨텍스트조차 잡지 못하고 종료하는 경우가 있으며, 2 차 검증의 여덟 개 동시 시작에서 나타나지 않은 것은 그때 몫의 크기가 모두 같고 할당이 점진적이었기 때문이다. 시작 간격을 1 초 이상 두거나 시작 직후의 메모리 부족을 한 번 재시도하면 해소된다.

세 결함 모두 회피 방법이 명확하고 비용이 크지 않으며, 이 가운데 두 번째와 세 번째는 상위 저장소에 보고할 가치가 있다.

4. 결론

4.1 검증 대상 가정에 대한 판단

1.2 에서 세운 가정은 성립한다. 물리 GPU 한 장은 여러 개의 격리된 뭉으로 나뉘고, 각 뭉의 VRAM 상한은 실제로 강제되며, 그 안에서 돌린 다섯 종의 실제 작업이 스무 번 모두 분할하지 않았을 때와 비트 단위로 같은 결과를 냈고, 어떤 이웃 작업도 다른 사용자의 결과를 틀리게 만들거나 그래픽카드에 장애를 일으키지 못했다. 미리 정해 둔 여섯 가지 중단 조건은 한 번도 발동하지 않았다.

여기에 처음 계획에 없던 확인이 두 가지 더해졌다. 그래픽카드 한 장을 아홉 명까지 분할해도 성능이 인원수에 반비례해 고르게 나뉘었고, 세션을 12 시간 유지해도 상한과 회계와 결과가 전혀 변하지 않았다. 동시 실행 결과는 교육 활용의 근거가 되고 장시간 유지 결과는 임대 방식의 전제가 되므로, 두 결과가 각각 선택한 구현 방식의 핵심 근거에 해당한다.

따라서 「구현 방식 검토 결과보고서」의 결론은 기술적으로 유지되며, 이제 이 검증이 밝힌 제약들을 구현에 반영하는 일이 남는다. 이하 4.2 부터 4.4 까지가 그 제약이고, 성격에 따라 이미지를 어떻게 만들 것인가, 몇 명을 배정할 수 있는가, 무엇을 정책으로 정해야 하는가로 나누어 정리한다.

4.2 컨테이너 구성 요건

사용자에게 관리자 권한을 줄 수 없다. 상한은 `/etc/ld.so.preload` 에 읽기 전용으로 주입할 때만 지켜지고 그 파일을 고칠 수 있는 사용자에게는 제한이 적용되지 않으므로, 이는 상한이 성립하기 위한 조건이어서 선택의 여지가 없다. 대신 사용자가 `apt` 계열의 시스템 패키지를 설치할 수 없게 되므로, 필요한 패키지가 생겼을 때 관리자에게 요청해 이미지에 반영하는 경로를 반드시 마련해야 한다. 검증 결과 사용자 영역 패키지 설치와 Jupyter, TensorBoard 실행은 권한 없이도 모두 가능해 통상적인 사용에는 지장이 없으나, 시스템 패키지 설치를 요구하는 문의는 발생할 수밖에 없다.

홈 디렉터리를 어디에 두느냐가 그다음 문제다. Docker 가 기본으로 붙이는 임시 파일 시스템은 실행 권한이 없어 여기에 홈을 두면 컴파일된 확장을 포함한 패키지가 설치되는 되지만 실행되지 않으며, Jupyter 는 `pyzmq` 에서 TensorBoard 는 `grpc` 에서 각각 세그먼트를 매핑하지 못했다는 알아보기 어려운 오류로 죽는다. 세션마다 실행 가능한 볼륨을 홈으로 붙여야 하므로 저장소 설계를 세션 생성 절차와 떼어 놓고 정할 수 없다. 주입 파일의 배치에도 제약이 따르는데, `/etc/ld.so.preload` 를 지닌 컨테이너는 GPU 라이브러리가 함께 주입되지 않으면 어떤 프로세스도 시작하지 못하므로 CPU 전용 작업을 지원한다면 이미지 계열을 분리해야 한다.

이미지는 두 종류로 나눈다. 실습과 일반적인 연구에는 6.3 GB 의 실행용 이미지로 충분하고 CUDA 확장을 직접 컴파일해야 하는 사용자에게는 21.3 GB 의 개발용 이미지가 필요한데, 두 종류를 모든 노드에 두면 노드마다 27.6 GB 를 차지하므로 개발용 이미지를 일부 노드에만 두고 해당 요청을 그쪽으로 보내는 방식도 검토할 만하다. 어느 쪽이든 교내망을 통한 이미지 배포 자체가 별도의 과제가 된다. 이미지 수준에서 할당자 설정도 함께 고정해 사용자가 `cudaMallocAsync` 백엔드를 켜지 못하게 하며, 기본 할당자에서는 CUDA 그래프와 컴파일 최적화가 모두 정상 동작하므로 읽는 기능이 없다.

4.3 자원 배정 계산의 근거

배정 계산의 출발점은 그래픽카드의 총 메모리가 아니라 드라이버가 예약한 몫을 제외한 여유량이다. 검증 장비에서 총량은 16 376 MiB 이지만 실제로 내줄 수 있는 양은 15 904 MiB 였고, 총량을 기준으로 계산하면 그래픽카드 한 장마다 216 MiB 를 초과 배정하게 된다. 여기에 더해 몫 하나에 대해 세 가지 값을 구분해야 하는데, 설정값은 우리가 지정하는 숫자이고 사용자가 실제로 쓸 수 있는 양은 거기서 256 MiB 를 뺀 값이며 그래픽카드에서 실제로 소모되는 양은 설정값에 32 MiB 를 더한 값이므로, 배정 계산에는 여유를 두어 64 를 더한 값을 쓴다. 사용자에게 표시할 숫자는 설정값이 아니라 실제 가용량이며, 그러지 않으면 배정량과 실제 사용 가능량이 달라 혼선이 생긴다.

등급은 목표 인원에서 역산해 정한다. 인원 N 에 대해 등급과 64 의 합에 N 을 곱한 값이 여유량 이하가 되는 가장 큰 64 정렬 값을 택하며, 이 규칙으로 계산한 여섯 등급이 3.3 의 표 2 의 거듭제곱으로 잡으면 소형과 중형 등급에서 각각 한 자리씩 줄어든다. 다만 계산에 들어가는 상수는 그래픽카드마다 다시 측정해야 하는데, 컨텍스트 비용 290 MiB 와 HAMi 내부 계상 250 MiB 와 초과분 32 MiB 가 모두 이 그래픽카드와 이 드라이버에서 나온 값이어서 세대가 다른 그래픽카드에서 같으리라는 보장이 없기 때문이다. 계산 규칙 자체는 HAMi 의 회계 구조에서 나오므로 유지되고 숫자만 움직이므로, 새 노드를 편입할 때 상수를 산출하는 절차를 필수 단계로 두면 된다. 이를 위한 자동화 도구는 작성해 두었다(별첨 5).

CPU 코어 수는 메모리와 별개로 인원을 제한한다. 여덟 명이 동시에 돌 때 데이터 적재 워커 서른두 개가 8 코어를 가득 채웠는데, GPU 쪽 처리량에는 영향이 없었지만 시작 시간이 느려지므로 노드의 수용 인원은 메모리로 계산한 값과 CPU 로 계산한 값 중 작은 쪽이다. 기증받는 장비의 코어 수를 조사 항목에 넣어야 한다.

작은 등급에서는 사용자가 쓸 수 있는 프로세스가 사실상 하나로 제한된다. 상한이 몫 단위로 걸리고 CUDA 컨텍스트 하나가 예산에서 250 MiB 를 차지하므로 프로세스를 추가할 때 마다 그만큼이 사용자 자신의 몫에서 빠지는데, 여덟 명 등급인 1920 MiB 는 내부 가용량이 1 664 MiB 이고 ResNet-18 실습이 1 380 MiB 안팎을 쓰므로 남는 여유가 284 MiB, 곧 컨텍스트 하나를 겨우 넘는다. 학생이 노트북을 두 개 열면 두 번째 커널이 컨텍스트를 확보하지 못한다. 이 제약은 실습 환경에서 두 번째 노트북을 실행하는 시점에 드러나므로 실습 등급을 한 단계 올리거나 세션당 커널을 하나로 제한하고 그 사실을 명시하는 두 가지 중 하나를 미리 정해 두어야 한다.

4.4 운영 정책 요건

실행기는 컨테이너 시작에 1초 이상 간격을 두고, 그와 별개로 시작 직후의 메모리 부족을 한 번 재시도한다. 간격만으로는 부하 상황에서 분리가 보장되지 않아 둘을 함께 적용하며, 여덟 명 기준으로 늘어나는 시간은 8초 정도로 무시할 수 있다. 작업이 실패했을 때는 세 종류를 구분해 알려야 하는데, 컨테이너가 호스트 메모리 부족으로 죽은 경우와 외부에서 종료된 경우와 몫이 모자라 GPU 메모리 부족이 난 경우가 호스트에서 각각 다르게 관찰되기 때문이다. 특히 마지막 경우를 “작업이 실패했습니다”로 표시하면 사용자가 할 수 있는 일이 없으므로, “배정받은 GPU 메모리가 부족합니다”로 알려 스스로 조치하게 한다.

연산 제한은 두 가지 용도로 구분해 제공한다. 제한을 걸지 않으면 시간 분할이 커널 실행 시간을 기준으로 나누므로 무거운 모델을 실행하는 학생이 가벼운 모델을 실행하는 학생보다 많은 연산을 배정받고, 성적이 걸린 과제에서 이는 공정성 문제가 되어 연산 제한이

필수가 된다. 반면 제한이 걸린 작업은 그래픽카드가 비어 있어도 대기 상태를 유지하므로 연구용 세션에 기본으로 적용하면 자원을 낭비한다. 따라서 성적 평가용 좌석에는 고정 배분으로, 연구용 세션에는 경합 시의 우선순위 수단으로만 적용하는 두 가지 모드를 두고, 어느 쪽에서도 이 값을 성능의 백분율로 표시하지 않는다.

관측은 호스트 쪽에서 수행한다. 컨테이너 안의 NVML은 가상화된 값을 돌려주어 사용량 집계와 과금 근거로 쓸 수 없으므로, 실행기가 호스트에서 표본을 뜨고 프로세스를 컨테이너에 대응시키는 방식을 유지한다. 여기에 더해 작업의 메모리 사양을 평가 단계까지 포함해 산정해야 하는데, 학습과 검사와 체크포인트 중 메모리를 가장 많이 쓰는 것은 대개 학습이고 정점도 시작 몇 초 안에 도달하지만 학습보다 큰 덩어리를 한 번에 요구하는 단계가 있으면 실행이 거의 끝난 시점에 실패하기 때문이다. GPT-2를 평가 배치만 크게 잡아 실행했을 때 전체의 94%를 지난 지점에서 실패했으므로, 사용자에게는 평가 배치를 학습 배치보다 크게 잡지 말 것을 안내한다. 상한에 가까워진 작업을 미리 알리는 기능도 실행기에 넣어 두어 뭉의 90%와 97% 시점에 경고를 남기고 남은 시간을 추정하며, 점진적으로 늘어나는 경우에는 수 초에서 수십 초의 여유를 준다. 단계 전환에서 한 번에 증가하는 경우는 예측할 수 없지만 뭉의 95%를 오래 유지하는 상태 자체가 신호가 된다.

4.5 활용 시나리오별 합의

교육 활용에서는 앞선 보고서가 주장한 규모가 계산으로 확인된다. 실습용 등급에서 그래픽카드 한 장에 여덟 명을 배정할 수 있고 성능도 여덟 명이 고르게 나누므로 열두 장이면 아흔 명 이상을 동시에 수용하며, BERT 규모의 과제라도 그래픽카드당 네 명이므로 마흔여덟 명이다. 마흔 명 한 학급은 두 경우 모두 여유가 있다.

수강생 전원이 같은 조건에서 작업한다는 점도 확인했다. 모든 학생이 같은 크기의 메모리와 같은 뭉의 연산을 강제로 배정받으므로 어떤 장비를 배정받았는지가 결과에 영향을 주지 않으며, 성적이 걸린 과제에서 이는 편의를 넘어 공정성의 요건에 해당한다. 그래픽카드를 통째로 배정하는 방식으로는 원리상 달성할 수 없고, 다만 4.4에서 적었듯 연산 제한을 함께 걸어야 성립한다.

한 학급이 동시에 접속하는 상황도 문제가 되지 않는다. 여덟 개를 같은 순간에 시작해도 13초 안에 전부 작업에 들어가며 시작 간격을 두는 편이 오히려 느리지만, 4.4의 시작 간격 조치는 경합 회피를 위한 것이므로 별개로 적용한다. 주의할 점은 4.3의 마지막 항목으로, 실습용 등급에서 학생 한 명이 쓸 수 있는 CUDA 프로세스가 사실상 하나이므로 노트북 여러 개를 동시에 여는 사용 방식은 이 등급에서 지원되지 않는다.

연구 활용에서는 세션을 장시간 유지할 수 있는지가 임대 방식의 전제였고, 12시간 동안 어떤 값도 변하지 않았으므로 메모리 계층에서 오는 제한이 없다. 세션 길이를 제한한다면 그것은 자원 회수 정책의 문제이지 기술적 이유에서 나오지 않는다.

그래픽카드를 나눠 쓰는 대가는 속도다. 여덟 명이 함께 쓰면 각자의 작업은 여덟 배 느려지며, 그래픽카드 전체의 처리량이 99% 유지된다는 것은 낭비 없이 나눈다는 의미일 뿐이다. 짧게 실행해 보고 수정하기를 반복하는 작업 방식은 그래픽카드가 비어 있을 때 유리하므로, 연구용 세션과 실습용 좌석의 밀도를 같게 둘 이유가 없다. 직접 커널을 컴파일하는 연구자는 개발용 이미지로 지원하며, 권한 없이도 컴파일과 실행이 되고 상한도 지켜지므로 기술적 제약은 없고 비용은 노드가 감당해야 할 15GB의 추가 용량뿐이다.

4.6 미검증 사항과 대응 방침

여러 장에 걸친 작업은 장비가 한 장뿐이라 두 가지를 확인하지 못했다. 컨테이너 하나가 서로 다른 물리 장치에 각각 독립적인 상한을 갖는지, 그리고 두 장에 걸친 분산 학습이 정확한 결과를 내는지이며, 장치별 상한 설정은 0 번 장치 형식만 실제로 사용해 보았다.

다만 한 장에서 확인할 수 있는 부분에서는 문제가 발견되지 않았다. 가로채기 계층이 집합 통신 자체에 영향을 주지 않고 NCCL의 통신 버퍼도 상한 안에서 계산되므로, 남은 것은 장치별 독립성과 분산 학습의 정확성뿐이고 그래픽카드 두 장짜리 장비를 한 시간만 쓸 수 있으면 마무리된다. 그 전까지는 여러 장을 쓰는 작업에 그래픽카드를 통째로 배정하고 분할하지 않는 것을 기본 방침으로 삼는다. 분산 학습을 수행하는 장비는 이미 자원을 대부분 사용하고 있어 추가로 분할할 이익이 크지 않고, 이 방침으로 두면 미검증 항목이 아무것도 막지 않으므로 나중에 확인이 끝나면 기능을 추가하는 문제가 된다.

Pascal 이전 세대 그래픽카드도 장비가 없어 확인하지 못했으나 이 항목은 성격이 다르다. HAMi 자체는 드라이버 440 이상이면 모든 NVIDIA 그래픽카드를 지원한다고 밝히고 있어 제약의 원인이 아니고, 실제 제약은 CUDA 13 이 Maxwell 과 Pascal 과 Volta 지원을 중단한 데서 오므로 검증에 쓴 이미지는 GTX 10 시리즈에서 애초에 실행되지 않는다. 따라서 이 항목의 답은 내년 여름 장비 조사에서 나온다. Turing 이전 그래픽카드를 받아들이면 CUDA 12 계열 이미지를 별도로 유지해야 하고 노드에 따라 사용자 환경이 달라져 모든 학생에게 같은 환경을 준다는 교육 활용의 근거가 노드 경계에서 성립하지 않으며, 받지 않으면 검증 가능한 장비 일부를 포기한다. 어느 쪽을 택하든 구형 그래픽카드의 수용 기준은 장비 조사에 착수하기 전에 정해 두어야 한다.

여러 노드에 걸친 운영은 세 차례의 검증이 모두 한 대 안에서 이루어져 다루지 못했다. 노드 등록과 배치와 상태 감시는 구현의 문제이지 불확실성이 아니지만, 검증받은 장비를 소유 연구실이 물리적으로 계속 보유한다는 점에서 오는 문제가 하나 있다. 소유자가 재부팅하거나 장비를 회수하면 그 위에서 실행 중이던 세션이 예고 없이 중단되고 상용 클라우드에는 대응하는 사례가 없으므로, 최소한 어느 노드든 사라질 수 있다고 전제하고 사용자 데이터는 노드 바깥에 두는 설계가 필요하다.

그 밖에 여러 사용자가 동시에 접근할 때의 저장소 성능은 저장소 구조를 정한 뒤에야 측정할 수 있어 미루었고, 화면이 연결된 그래픽카드는 유휴 사용량이 변동하므로 노드 편입 기준에서 다루는 편이 낫다. 열두 시간을 넘는 보유는 확인하지 않았으나 열두 시간 동안 움직인 값이 하나도 없었으므로 위험은 낮다고 본다.

4.7 종합 판단

기술적 타당성 검증은 마무리되었다고 판단한다. 선택한 구현 방식이 기대고 있던 가정을 확인했고, 확인 과정에서 우려했던 항목 몇 가지는 오히려 예상보다 견고하게 드러나, 정적으로 링크한 바이너리도 상한을 우회하지 못하고, 통신 라이브러리의 버퍼도 예산 안에서 계산되며, 세션을 12 시간 유지해도 어떤 값도 변하지 않는다. 찾아낸 결함 세 가지 역시 모두 회피 방법이 명확하고 비용이 작아, 주입 방식을 바꾸고 할당자 설정을 고정하고 시작에 간격을 두면 각각 해소된다.

확정된 제약 열몇 가지는 무엇을 만들 수 있는가보다 어떻게 만들 것인가를 정한다. 사용자에게 권한을 주지 않고, 흐름 실행 가능한 볼륨에 두고, 상수를 노드마다 다시 재고, 등급을 인원에서 역산하고, 성적용 좌석에 연산 제한을 걸어야 하지만, 어느 것도 설계를 바꾸지 않으며 전부 구현 단계에서 반영하면 된다. 남은 미검증 항목은 그래픽카드 두 장짜리 장비

에서 한 시간이면 끝나는 확인 하나와 애초에 이 검증의 범위 밖이었던 다중 노드 운영뿐이다. 두 장 검증은 기본 방침을 정해 두어 구현을 막지 않도록 했고, 다중 노드 운영은 불확실성이라기보다 아직 만들지 않은 부분에 해당한다.

요컨대 여름 개발 TF가 무엇을 만들어야 하는지에 대한 기술적 불확실성은 해소된 것으로 보인다. 이후의 과제는 검증에서 확정된 제약을 반영해 실제 시스템을 구현하는 일이며, 그 출발점이 되는 실행기는 세 차례의 검증을 거치며 이미 작성해 두었다.

위와 같은 검증 결과와 그로부터 도출한 제약들이, 이후 구현 단계의 판단에 참고가 되기를 바란다.

별첨

별첨 1. 검증 환경 및 소프트웨어 구성

호스트

항목	값
GPU	NVIDIA GeForce RTX 4080 SUPER, 16 376 MiB, CC 8.9, PCIe gen3, 320 W
유틸리티 사용량	17 MiB (디스플레이 미연결), 드라이버 예약 455 MiB
드라이버	580.178.04 (CUDA 13.0)
CPU / RAM / 디스크	Intel i7-9700KF 8 코어, 31 GiB, LVM 루트 935 GB
OS / 커널	Ubuntu 22.04.5 LTS / 5.15.0-190-generic
컨테이너 런타임	Docker 29.1.3, overlay2, cgroup v2, NVIDIA Container Toolkit 1.20.0

HAMi-core

커밋 f01e9f23fc6ab251d2a7fee8987279f16b08afc8 (2026-08-31). 저장소에 릴리스 태그가 없어 커밋을 고정했다. 상위 저장소의 make build-in-docker 절차를 그대로 사용해 libvgpu.so를 생성했다.

소스에서 추출한 환경변수는 다음과 같다.

변수	의미
CUDA_DEVICE_MEMORY_LIMIT / <i><i></i>	장치별 메모리 상한. g/m/k 접미사 또는 바이트. 장치별 형식이 우선한다
CUDA_DEVICE_SM_LIMIT / <i><i></i>	연산 사용률 백분율, 기본 100
CUDA_DEVICE_MEMORY_SHARED_CACHE	회계용 공유 메모리 파일 경로, 기본 /tmp/cudevshr.cache
LIBCUDA_LOG_LEVEL	로그 수준 0-4

변수	의미
GPU_CORE_UTILIZATION_POLICY	연산 제한기 강제/해제
ACTIVE_OOM_KILLER	해석은 되나 해당 커밋에서 호출되지 않는 죽은 코드

메모리 제한은 cuMemAlloc_v2, cuMemAllocManaged, cuMemAllocPitch_v2, cuMemAllocAsync, cuMemCreate 를 가로채 추적 사용량과 요청량의 합을 상한과 비교하는 방식이고, NVML 질의도 가로채 가상화된 값을 돌려준다. 연산 제한은 장치별 토큰 버킷을 커널 실행이 소모하고 감시 스레드가 채우는 구조로, 버킷이 비면 실행 스레드가 nanosleep 으로 대기한다.

컨테이너 사양

모든 실행은 --gpus device=0, --shm-size 2g, --network none, 비루트 uid, 작업 디렉터리 읽기 전용 마운트를 공통으로 사용했다. 분할 실행에는 libvgpu.so 를 읽기 전용으로 마운트하고 LD_PRELOAD 또는 /etc/ld.so.preload 와 상한 변수를 지정했다. 비교를 위해 라이브러리를 붙이지 않은 상태, 붙이되 상한을 걸지 않은 상태, 상한까지 건 상태의 세 조건을 구분해 사용했다.

검증 작업

식별자	모델 / 데이터	구성	기준 결과
resnet18	ResNet-18 / CIFAR-10	3 epoch, batch 128, fp32	acc 0.825, 3 853 img/s
resnet50	ResNet-50 / Imagenette-320	3 epoch, batch 64, 224px	acc 0.499, 331 img/s
bert	BERT-base / GLUE SST-2	1 epoch, batch 32, seq 64	acc 0.906, 491 ex/s
gpt2	GPT-2 / WikiText-2	1 epoch, batch 4, 512 블록	PPL 49.88 – 21.95
sd	Stable Diffusion 1.5	512×512, 25 step, fp16	1.20 img/s

데이터와 가중치는 측정 전에 모두 내려받아 두었고 측정 중 컨테이너는 네트워크 없이 실행했다.

별첨 2.1 차 검증 상세 결과

상한 강제

설정값	내부 가용	호스트 실측 점	초과분
2 GiB	1 792 MiB	2 082 MiB	+34
4 GiB	3 840	4 128	+32
8 GiB	7 936	8 224	+32

컨텍스트만 유지하는 프로세스의 물리 사용량은 라이브러리 유무와 무관하게 290 MiB 였고, HAMi 가 추가로 쓰는 GPU 메모리는 0 이며 호스트 메모리는 약 4 MiB, CPU 는 감시 스레드 몫으로 코어의 약 1 % 였다.

분할 상태의 작업 정확성

작업	기준 정점	최소 통과 몫	정확성	처리량 (기준 - 주입 - 분할)
resnet18	1 672 MiB	1 728 MiB	비트 일치	3 853 - 3 821 - 3 825 img/s
resnet50	6 662	6 720	비트 일치	330.8 - 331.5 - 330.5
bert	3 654	3 712	비트 일치	490.8 - 489.3 - 488.8
gpt2	6 730	6 784	PPL 일치	19 580 - 19 555 - 19 521 tok/s
sd	4 576	4 608	이미지 해시 일치	1.200 - 1.194 - 1.195

측정한 정점에는 캐싱 할당자가 쥐고 있다가 압박 시 반환하는 양이 포함되어 있어 실제 하한은 정점보다 5~25 % 낮다. 정점보다 512 MiB 낮게 잡았을 때 resnet18 은 정상 완료했고 bert 는 6.3 초 만에 깨끗한 메모리 부족으로 종료했다.

두 사용자 동시 실행

조합	지연 A / B	동시 실행 대 순차 실행
resnet18 + resnet18	1.98 / 1.95	0.996
resnet18 + bert	1.88 / 1.97	1.02
bert + gpt2	1.81 / 1.19	1.06
resnet50 + sd	1.06 / 3.41	1.15
gpt2 + sd	1.03 / 2.92	1.11
resnet50 + gpt2	1.96 / 1.57	1.03

악의적 이웃에 대한 격리

resnet18 을 대상 작업으로 두고 이웃을 바꿔 가며 확인한 결과, 이웃의 동작과 무관하게 대상 작업의 결과는 비트 단위로 정확했다.

이웃의 동작	결과	대상 작업 지연
메모리 부족 반복 (75 초간 1096 회)	정상 종료	1.04 배
3 초짜리 커널 연속 실행	정상 종료	1.95 배
실행 중 강제 종료	exit 137	1.31 배
라이브러리 없이 그래	정상 종료	1.04 배

이웃의 동작	결과	대상 작업 지연
픽카드 전체 점유		

연산 제한 반응

설정	큰 행렬곱	작은 커널 다수	ResNet-18 학습
100	1.00	1.00	1.00
75	0.97	0.89	—
50	0.92	0.96	0.54
25	0.74	0.54	0.25
10	0.32	0.21	—

실행기 연속 운전

3.01 시간, 34 회차, 204 건 제출 및 완료, 전건 정확. 회차마다 용량이 원래 값으로 복귀했고 3 시간 뒤 그래픽카드 사용량은 시작 전과 같은 16.9 MiB, 남은 프로세스와 Xid 는 각각 0 이었다.

별첨 3.2 차 검증 상세 결과

인원수에 따른 성능 분배 (ResNet-18, 1 728 MiB 몫)

인원	지연 최소 / 최대 / 평균	합산 처리량	편차	호스트 CPU 평균	점유 메모리
1	1.00 (42.6 초)	1.000	—	26 %	1.7 GB
2	2.00 / 2.01 / 2.00	0.971	1.002	40 %	3.4 GB
4	3.86 / 3.92 / 3.90	0.978	1.016	68 %	6.7 GB
6	5.67 / 5.83 / 5.78	0.985	1.028	87 %	10.1 GB
8	7.45 / 7.75 / 7.67	0.993	1.040	99 %	13.4 GB
9	8.30 / 8.71 / 8.62	0.991	1.049	99 %	15.1 GB
10	아홉 정상, 열째 시작 시 실패	0.991	—	99 %	15.9 GB

여덟 명 이상에서 CPU 가 포화하지만 워커를 하나로 줄인 대조 실험에서도 곡선이 같아, GPU 쪽 분배는 CPU 경합의 영향을 받지 않는다.

성격이 다른 작업의 혼합

구성	큰 작업 지연	작은 작업 지연
ResNet-50 + ResNet-18 다섯	3.31 배	5.85~5.97 배
BERT + ResNet-18 여섯	6.39 배	6.41~6.65 배

동시 시작

방식	초기화 실패	CUDA 컨텍스트 생성	전원 학습 시작
여덟 개 동시	0	3.4~3.7 초	12.8 초
2 초 간격	0	3.0~17.6 초	28.2 초

열여섯 건을 동시에 제출한 경우 실행기는 메모리가 허락하는 여덟 건만 실행했고 배정 총량이 용량을 넘긴 표본은 없었다.

등급별 최대 수용 (여유량 15 904 MiB, 배정 계산 = 설정값 + 64)

인원	설정값	내부 가용	인원 × 배정	잔여
8	1 920	1 664	15 872	32
6	2 560	2 304	15 744	160
4	3 904	3 648	15 872	32
3	5 184	4 928	15 744	160
2	7 872	7 616	15 872	32
1	15 808	15 552	15 872	32

가득 채워 확인한 결과 BERT 네 개는 정점 14 657 MiB, Stable Diffusion 세 개는 13 764 MiB, ResNet-50 과 GPT-2 조합은 13 425 MiB 로 모두 정상 완료했다.

무권한 사용자의 동작

행위	홈이 실행 불가 불륨일 때	홈이 실행 가능 불륨일 때
사용자 영역 패키지 설치	가능	가능
설치한 명령 실행	실패	가능
Jupyter Lab 실행	실패 (pyzmq)	90 초 내 기동
TensorBoard 실행	실패 (grpc)	가능
시스템 패키지 설치	불가	불가
환경변수 제거 후 상한 초과	거절	거절
로더 사본을 통한 우회	거절	거절

단계별 메모리 사용 (살아 있는 텐서 기준, MiB)

작업	학습	평가	체크포인트
resnet18	963	454	194
resnet50	5 614	1 017	342
bert	2 974	2 017	1 753
gpt2	5 523	3 985	2 514

평가와 체크포인트가 학습을 넘긴 사례는 없었다. 다만 평가 배치를 학습 배치보다 크게 잡은 GPT-2는 1.54 GiB 덩어리를 한 번에 요구해 전체의 94 % 지점에서 종료했다.

별첨 4.3차 검증 상세 결과

집합 통신 라이브러리

확인 항목	결과
분할 상태에서 NCCL 초기화	프로세스 그룹 형성, 집합 연산 정상, DDP 손실값이 미분할 기준과 동일
통신 버퍼의 계상	계상됨. cudaMalloc 경로와 cuMemCreate 경로 모두
뭉칠 채운 뒤 초기화	4 GiB 선에서 거절. 대조군은 5 GB 까지 무제한 증가
gloo 백엔드	정상
한 장에 두 랭크	NCCL 자체가 거절 (HAMi 와 무관)

12 시간 보유

경과	강제 상한	학습 결과 해시	정확도	처리량	새 프로세스가 본 여유
0 시간	3 456	ce935317dcc2	0.6224	3 787	3 469
1 시간	3 456	동일	0.6224	3 800	3 469
2 시간	3 456	동일	0.6224	3 806	3 469
4 시간	3 456	동일	0.6224	3 802	3 469
6 시간	3 456	동일	0.6224	3 775	3 469
8 시간	3 456	동일	0.6224	3 778	3 469
10 시간	3 456	동일	0.6224	3 792	3 469
11.8 시간	3 456	동일	0.6224	3 782	3 469

컨테이너의 호스트 메모리는 첫 작업에서 470 MiB 에서 781 MiB 로 오른 뒤 열두 시간 동안 고정이었고 CPU 는 평균 2 %였다. 73 회의 작업이 모두 완료되었으며 소요 시간은 첫 회 1.18 초 이후 0.2 초로 안정되었다. 두 컨테이너 종료 후 그래픽카드는 17 MiB 로 복귀했고 Xid 는 0 이었다.

이 회차에서 동시 시작 경합을 발견했다. 두 컨테이너를 같은 순간에 실행했을 때 한쪽이 첫 작업으로 2 GiB 를 요구하는 사이 다른 쪽이 그 사용량을 자기 뭉 안의 값으로 잘못 계산해 시작 2 초 만에 종료했다. 유지 중에 새로 실행한 컨테이너는 정상적으로 자기 뭉을 받았

고 앞선 회차의 여덟 개 동시 실행에서도 나타나지 않았으므로, 시작 시점에 한정된 경합으로 판단한다.

개발용 이미지

확인 항목	결과
무권한 사용자의 CUDA 커널 컴파일 및 실행	21.5 초, 결과 정상
확장이 드라이버 함수를 직접 호출한 할당 cudart 정적 링크 실행 파일	계상됨, 3 840 MiB 에서 상한 상한 유지 (드라이버 라이브러리는 동적 호출)
환경변수 제거 및 로더 사본 우회	모두 거절
이미지 용량	6.3 GB → 21.3 GB (+15.0 GB)

별첨 5. 그래픽카드별 상수 산출 절차

새 노드를 편입할 때 아래 세 상수를 다시 측정한다. 자동화 도구를 작성해 두었으며 검증 장비 기준 28 초가 소요된다.

1. **유틸리티 NVML 값** — 총량, 사용량, 여유량, 드라이버 예약량을 읽는다. 배정의 기준이 되는 것은 여유량이다.
2. **CUDA 컨텍스트 물리 비용** — 컨텍스트만 생성하는 프로세스를 미분할 상태와 분할 상태에서 각각 실행해 물리 사용량과 HAMi 내부 계상액의 차이를 얻는다.
3. **내부 가용량과 초과분** — 분할 상태에서 단계적 할당기를 돌려 실제로 쓸 수 있는 상한을 얻고, 호스트 실측값과 설정값의 차이로 초과분을 얻는다.

이 세 값에서 배정 규칙과 등급표가 계산된다. 계산 규칙 자체는 HAMi 의 회계 구조에서 나오므로 그래픽카드가 바뀌어도 유지되고 숫자만 움직인다. 반면 상한이 실재하고, 초과가 잡을 수 있는 예외로 나타나며, 상한이 프로세스가 아니라 뭉에 걸리고, 내부 NVML 이 가상화되는 것은 모두 사용자 공간 가로채기라는 구조에서 나오므로 하드웨어와 무관하다.

별첨 6. 미완료 검증 항목의 수행 절차

그래픽카드 두 장짜리 Turing 이후 세대 장비에서 한 시간 정도면 완료할 수 있다.

1. **장치별 상한의 독립성** — 컨테이너 하나에 두 장을 모두 붙이고 0 번에 2 GiB, 1 번에 6 GiB 를 설정한 뒤 컨테이너 안에서 장치별 총량이 각각 2 048 과 6 144 로 보이는지, 단계적 할당기가 각각 1 792 와 5 888 에서 멈추는지 확인한다. 지금까지 실제로 사용해 본 것은 0 번 형식뿐이다.
2. **두 장에 걸친 분산 학습** — 각 랭크를 4 GiB 뭉에 넣고 ResNet-18 분산 학습을 수행해 미분할 상태의 두 장 실행과 결과가 일치하는지 확인한다. 호스트 쪽에서 장치별 사용량을 관측해 어느 랭크도 배정된 뭉을 넘지 않는지, Xid 가 발생하지 않는지 함께 본다.

두 항목이 통과하면 여러 장을 쓰는 작업에도 분할을 적용할 수 있고, 실패하면 4.6 의 기본 방침대로 그래픽카드를 통째로 배정한다.

참고자료

- Project-HAMi/HAMi-core, <https://github.com/Project-HAMi/HAMi-core> (검증 사용 커밋 f01e9f23)
- Project-HAMi/HAMi, <https://github.com/Project-HAMi/HAMi>
- HAMi 지원 장치 목록, <https://project-hami.io/docs/userguide/device-supported>
- HAMi GPU 가상화 원리, <https://project-hami.io/docs/core-concepts/gpu-virtualization>
- NVIDIA CUDA Toolkit 12.9 릴리스 노트 (Maxwell·Pascal·Volta 지원 중단 예고)
- CIFAR-10, [uoft-cs/cifar10](https://github.com/ucbcsd/cifar10)
- Imagenette, [fast.ai](https://fast.ai/imagenette/)
- GLUE SST-2, [nyu-mll/glue](https://github.com/nyu-mll/glue)
- WikiText-2, [Salesforce/wikitext](https://github.com/Salesforce/wikitext)
- Stable Diffusion v1.5, [stable-diffusion-v1-5/stable-diffusion-v1-5](https://github.com/CompVis/stable-diffusion)
- 「GPU 공동 사용 플랫폼」 구현 방식 검토 결과보고서, 2026-08